Multi-Agent-Trading-Advisory-System

GitHub repository: https://github.com/jasonkong65/Multi-Agent-Trading-Advisory-System

Problems and Motivation:

Making trading decisions is not only about checking a stock price or asking a chatbot for advice. In real stock research, users need to collect market data, compare different signals, consider risk, review relevant information, and keep track of previous decisions. This process can be difficult for individual users because it is time-consuming and usually requires several different tools.

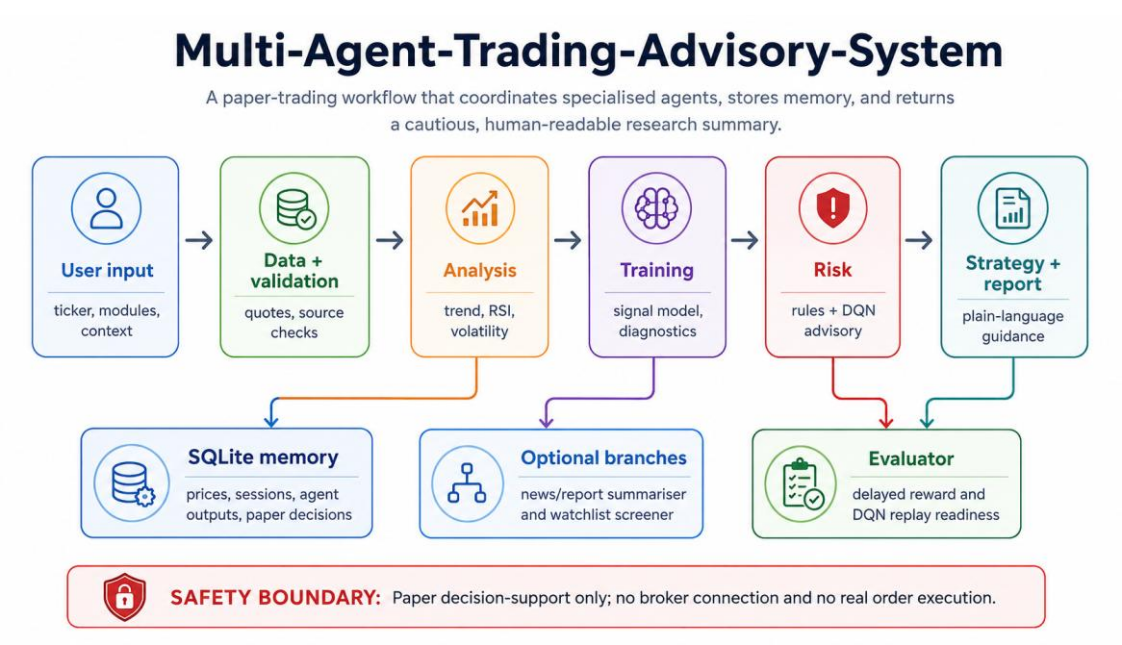
This project addresses this problem by building a Multi-Agent-Trading-Advisory-System for paper trading support. Unlike a simple chatbot, the system follows a structured workflow where different agents collect data, validate information, analyze signals, check risks, and evaluate past paper decisions. The system does not place real trades. Instead, it provides a clear recommendation, explanation, and risk warning, while keeping the final decision under human control.

Agentic Goal:

The goal of this system is to act as a trading consultation agent, not just a simple information chatbot. It observes market-related inputs, uses external tools or APIs to collect data, checks whether the data is reliable, analyses price movement and model signals, and then produces a conservative paper trading suggestion such as Buy, Sell, or Hold. Instead of giving an immediate one-step answer, the system breaks the task into several stages and coordinates different agents so that the final recommendation is supported by data validation, analysis, and risk control.

The system also uses memory to make the workflow more agentic. Each paper decision is stored in a SQLite database, allowing the system to review previous suggestions and later compare them with market outcomes. This delayed evaluation helps the system move beyond one-time responses and behave more like a decision-support agent. The human user always remains in control because the system only provides research support, explanations, and risk warnings, and it never places real trades.

System Design Diagram:



The Full workflow is on the Appendix: Full workflow evidence

Architecture Explanation:

The system is designed as a multi-agent trading consultation workflow rather than a simple chatbot. It follows a structured pipeline:

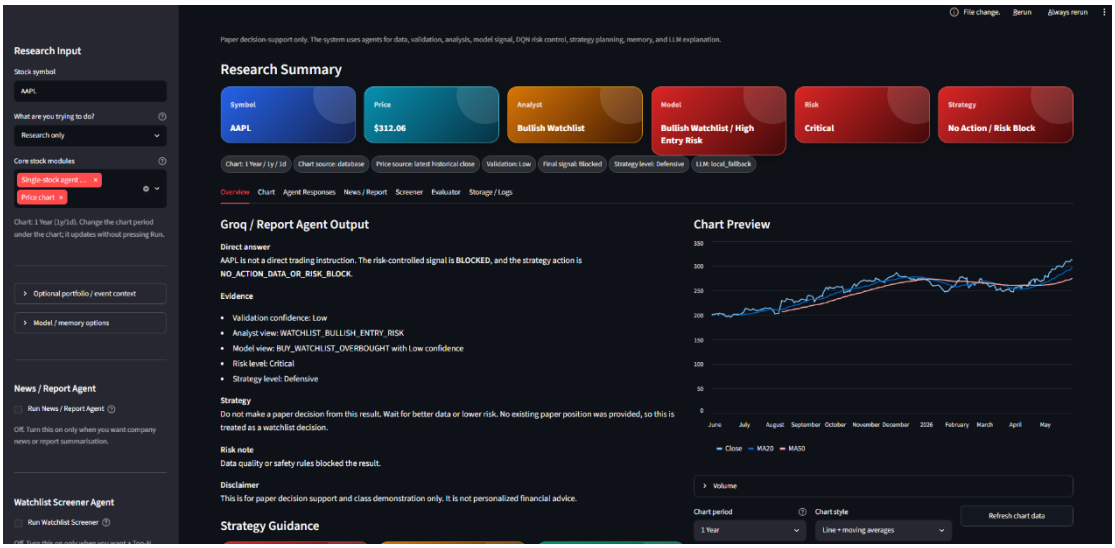
Data Agent → Validation Agent → Historical Data Agent → Analyst Agent → Training Agent → Risk Agent → Strategist Agent → LLM Report Agent

The Data Agent and Validation Agent first collect and check market data to make sure the information is usable and reliable. The

Historical Data Agent and Analyst Agent then examine past prices and extract key indicators such as trend, momentum, RSI, volatility, and volume behavior.

After the analysis stage, the Training Agent loads or trains the signal model and runs diagnostics to check whether the model is working properly. The Risk Agent applies strict safety rules together with a PyTorch DQN advisory layer, so risky or uncertain signals can be downgraded or blocked. The Strategist Agent then turns the controlled signal into a conservative paper trading plan, while the LLM Report Agent explains the final recommendation, reasons, risks, and confidence level in plain language. This architecture demonstrates agentic behavior because the system observes inputs, uses tools and models, checks risks, records decisions, and supports the user without executing real trades.

The System Works:



More System Works can be seen at Appendix: The Full System Works

The working system is shown through several main screens in the Streamlit interface. The Overview + Chart screen presents the final paper trading signal, colored summary cards, the Groq/LLM Report Agent explanation, strategy guidance, and a price chart with moving averages. This shows that the system does not only output a raw model result but turns the multi-agent workflow into a clear trading consultation report. Users can also change the chart period without rerunning the full agent pipeline, which makes the interface more efficient and easier to explore.

The Agent Responses, Screener, and Evaluator + Logs screens demonstrate the system's transparency, research support, and memory functions. Users can expand raw agent outputs to see how the recommendation was formed through data collection, validation, analysis, risk checking, and reporting. The Screener can scan a watchlist and rank candidates for further research without making risky investment promises. The Evaluator and Logs page stores paper decisions, agent outputs, UI sessions, and delayed reward information for future evaluation and DQN replay. Early N/A values are shown honestly when reward horizons are not yet complete.

Evaluation and Testing:

The final system was tested using both automated pytest tests and manual checks in the Streamlit interface. The tests covered important parts of the project, including imports, app helper functions, SQLite storage, workflow helpers, Risk Agent behavior, Training Agent basics, LLM fallback, and Screener logic. Local verification showed that all 16 tests passed, and the compile checks also passed. This confirms that the main components can run, connect correctly, and support the core workflow.

The testing also included both success and failure cases. In successful cases, the system could collect valid stock data, complete the full agent workflow, generate a paper trading signal, display agent outputs, save the decision into SQLite, and produce a readable LLM report. In failure or edge cases, such as missing market data, unreliable inputs, or an unavailable LLM service, the system used validation and fallback logic instead of crashing or giving overconfident advice. The Evaluator tab also correctly

shows N/A when delayed reward horizons are not complete, rather than inventing performance results. This provides evidence that the system was tested not only for normal workflows, but also for robustness when data, model outputs, or delayed reward information are incomplete.

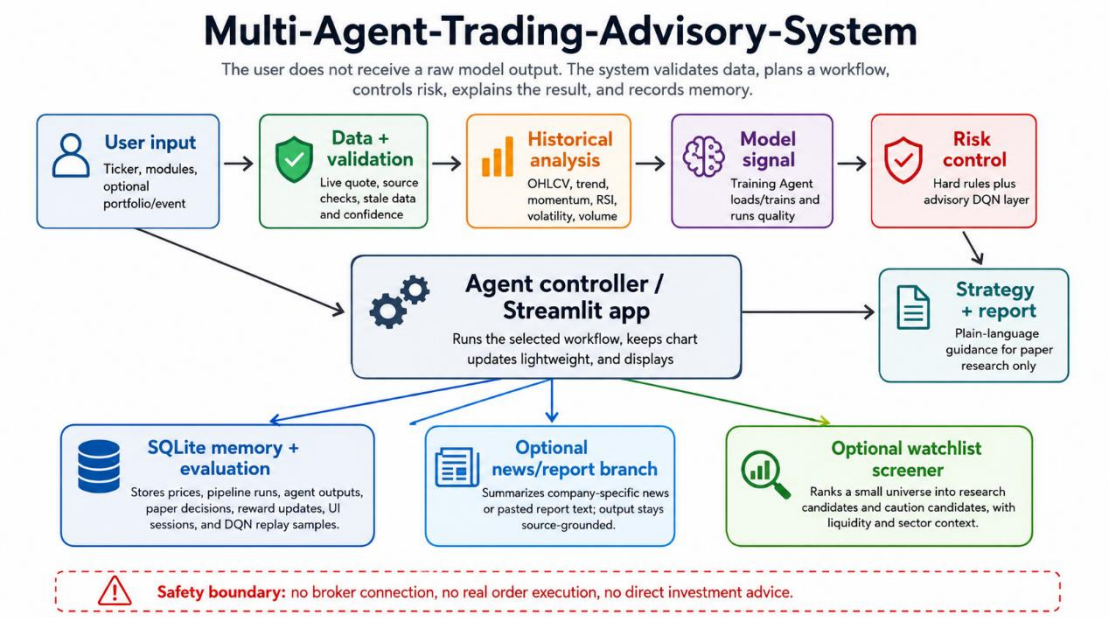
Critical Reflection:

This system works well as a small prototype of an intelligent software agent because it demonstrates key agentic features, including market perception, multi-step planning, API/tool use, SQLite memory, risk control, and delayed evaluation. It is more advanced than a simple chatbot because the final paper trading suggestion is produced through data validation, signal analysis, safety checks, and a clear explanation, rather than a one-step text response.

However, the system is still only a coursework prototype, so many functions and algorithms need further improvement. Free market data may be delayed or incomplete, and the DQN advisory layer needs many completed paper decisions before its learning becomes meaningful. To become a highly reliable trading advisory system, it would require stronger algorithms, larger and cleaner datasets, better backtesting, more computing resources, and stricter security controls. Therefore, this project is positioned as paper trading decision support only, not real trading or financial advice. Future improvements could include broader market data, user-level paper portfolios, a deployed database such as PostgreSQL, and stronger safety and evaluation mechanisms for larger-scale use.

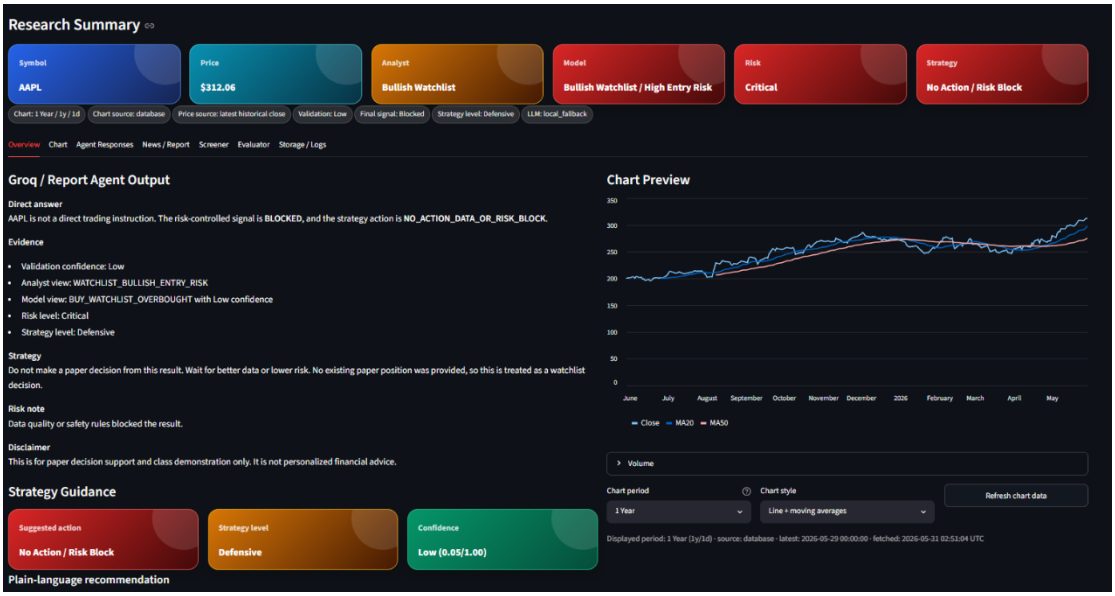
Appendix: larger implementation of evidence and detailed notes

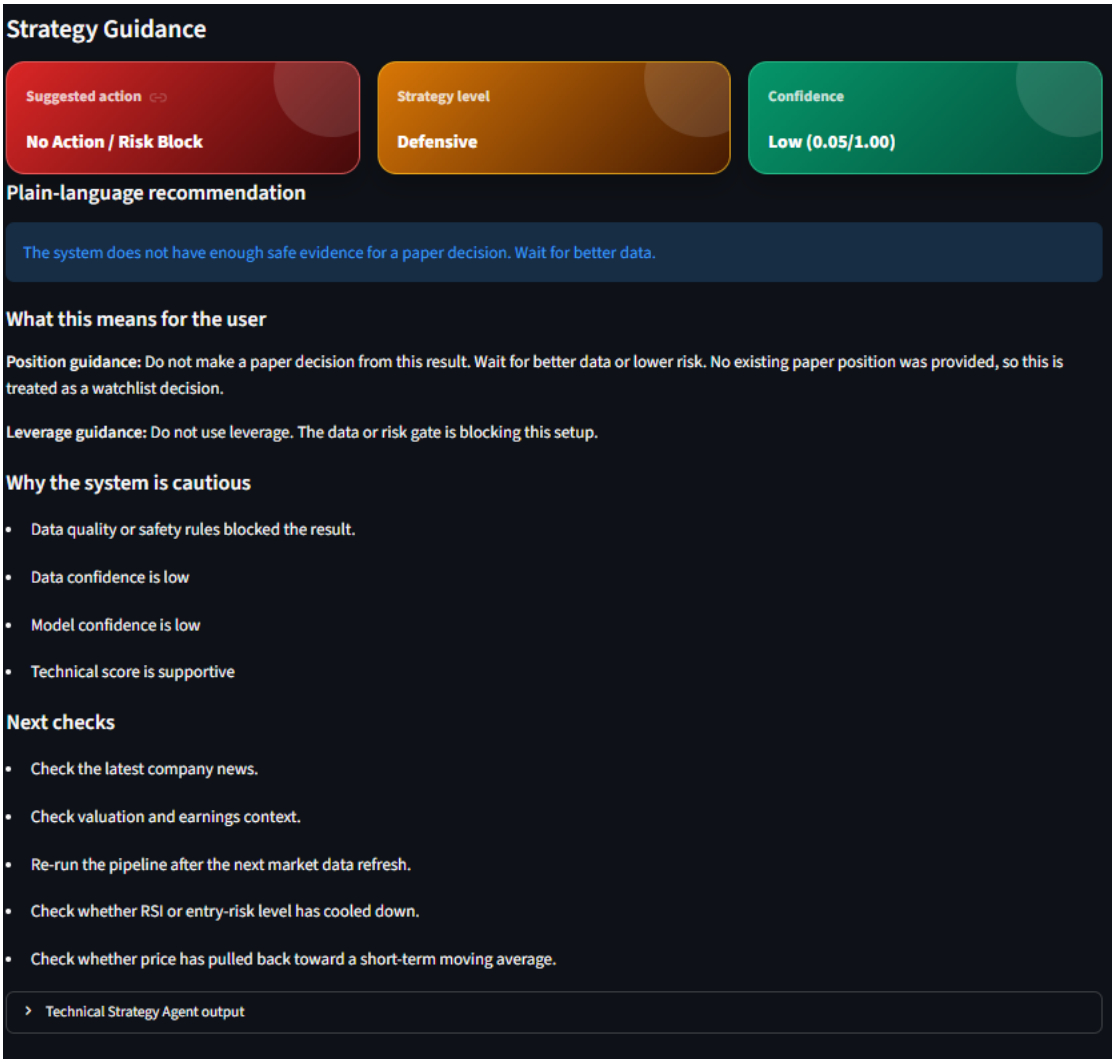
Full Workflow Evidence



The full workflow shows how the agent controller coordinates data collection, validation, historical analysis, model signal generation, risk control, strategy planning, reporting, and memory storage. The SQLite memory records pipeline runs, paper decisions, agent outputs, UI sessions, and information needed for delayed evaluation and DQN replay.

Overview Page



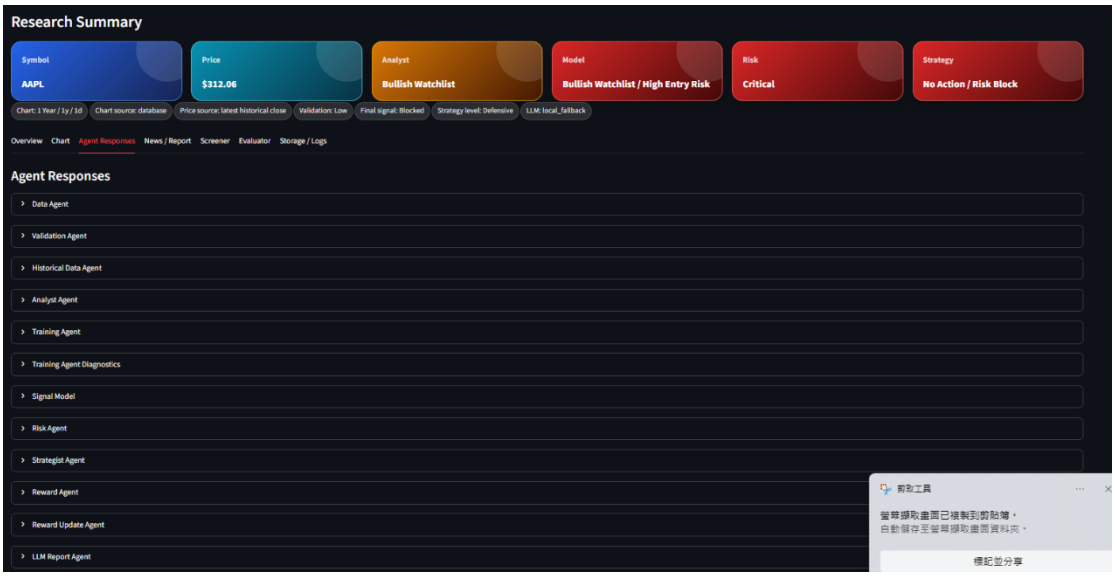


After the user enters a stock symbol for research, the system displays the research summary, strategy guidance, and live chart. This page provides the main user-facing result of the multi-agent workflow.

Chart Page

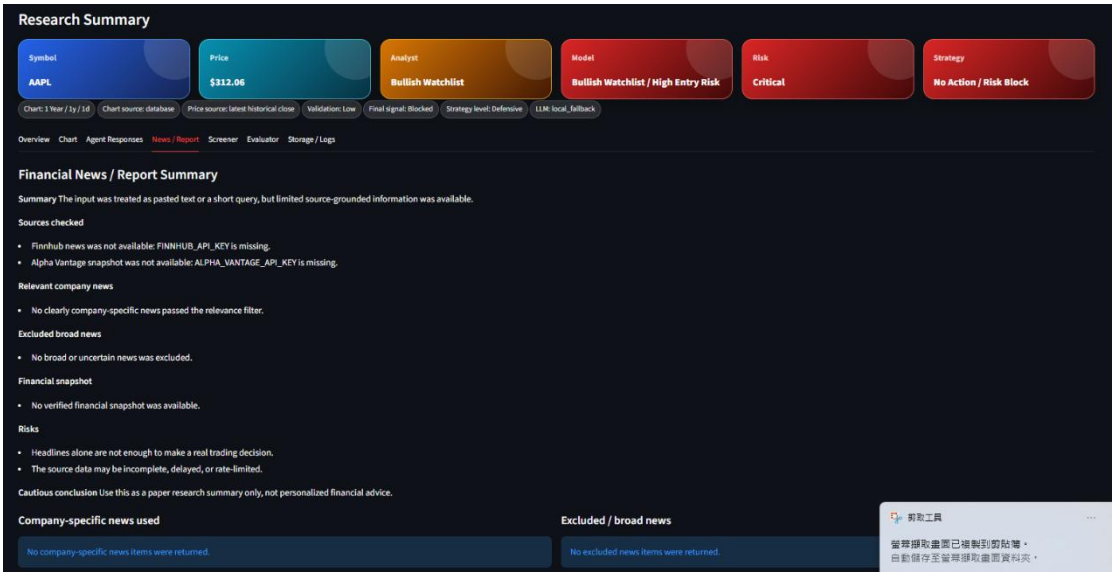
The chart page shows the live chart and chart data preview. Users can change the chart period without rerunning the full agent workflow, which keeps the interface lightweight and efficient.

Agent Responses Page



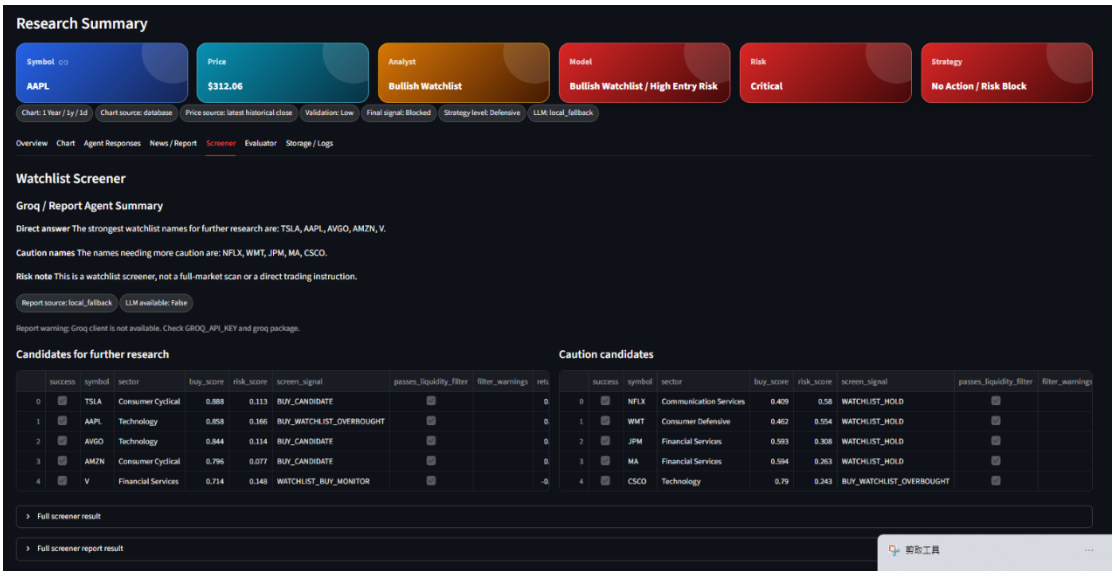
The Agent Responses page records the execution results of all agents and stores them locally. This allows the user to inspect the intermediate outputs and understand how the final recommendation was produced.

News / Report Page



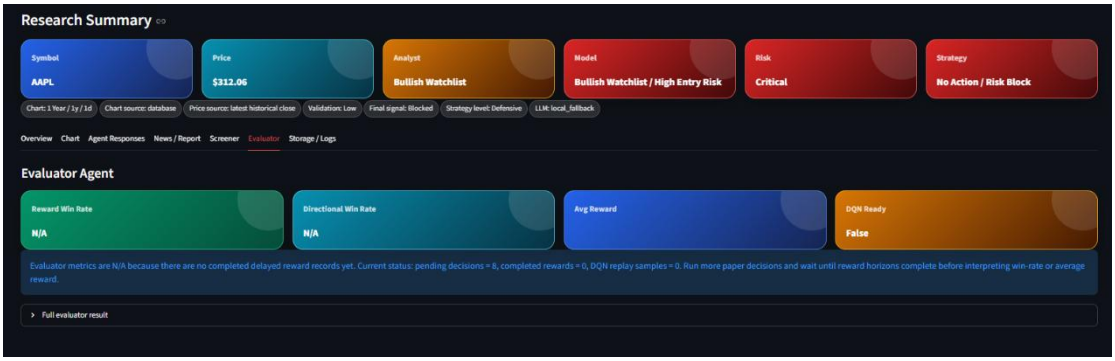
The News / Report page allows the user to input a stock and select the desired viewing mode, such as company news or financial report information. The system then provides a structured summary while keeping the output source grounded.

Screener Page



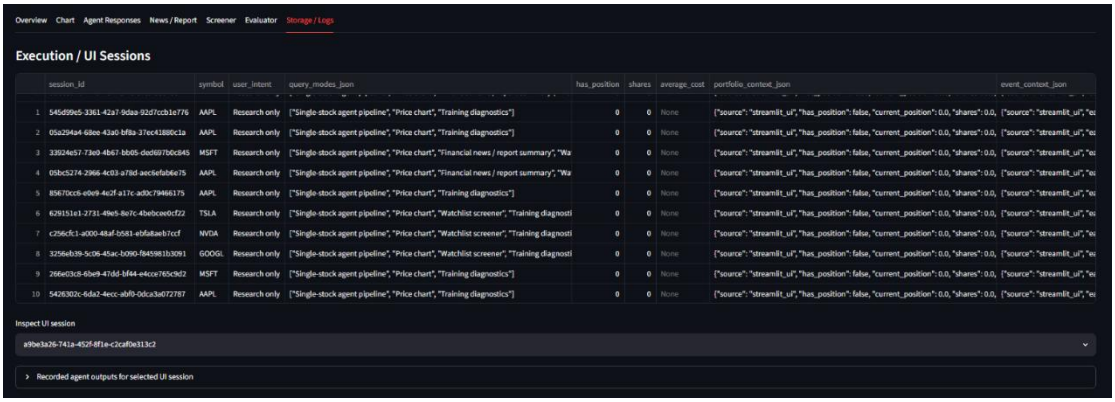
The Screener page scans a watchlist and separates symbols into research candidates and caution candidates. This helps the user identify stocks for further research without presenting the results as guaranteed investment opportunities.

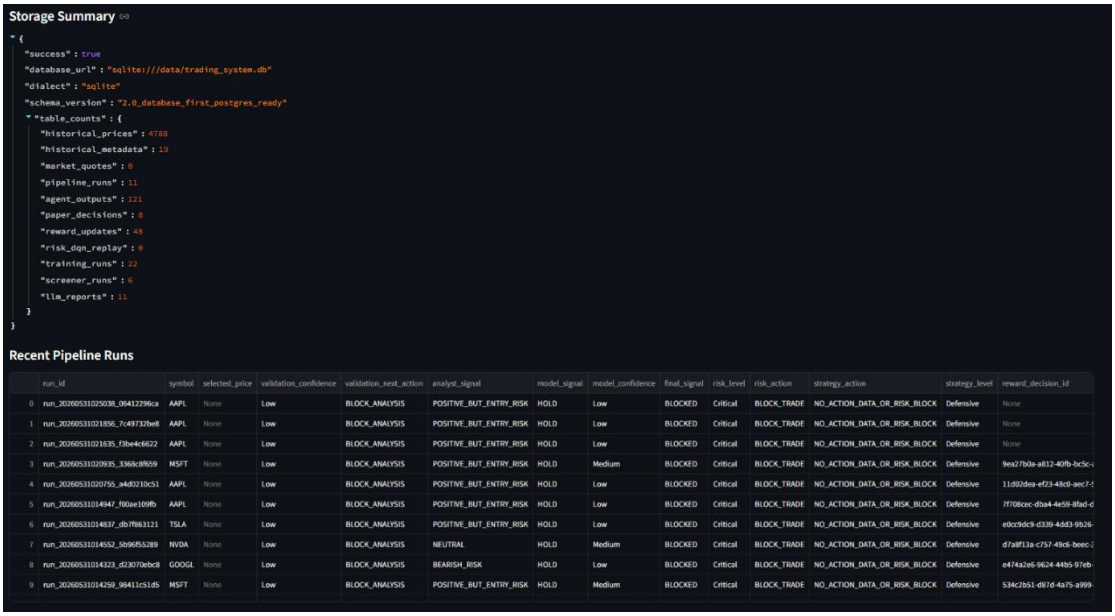
Evaluator Page



The Evaluator page shows delayed reward status, explains incomplete N/A outcomes, and indicates DQN replay readiness. This supports later evaluation once the paper trading reward horizon has been completed.

Storage / Logs Page





The Storage / Logs page shows recorded UI sessions and persistent SQLite memory. This demonstrates that the system stores agent outputs, pipeline runs, paper decisions, and evaluation-related data for future review.